Informe de Laboratorio 02

Tema: JavaScript

Nota

Estudiante	Escuela	Asignatura
Francis Willian Ayamamani Añasco fayamamani@unsa.edu.pe	Escuela Profesional de Ingeniería de Sistemas	Programación Web 2 Semestre: I Código: GUIA-PRLD-001

Laboratorio	Tema	Duración
02	JavaScript	04 horas

Semestre académico	Fecha de inicio	Fecha de entrega
2023 - A	Del 5 Junio 2023	Al 5 Junio 2023

1. Tarea

- Ejercicio 01: Utilizando JavaScript, genere un teclado aleatorio para Banca por Internet. Funcionalidad: Los números presionados generan una clave.



- Ejercicio 02: Utilizando JavaScript, genere una calculadora como la siguiente imagen. Cada vez que se presiona ENTER se guardan los datos involucrados en un pila.



2. MARCO CONCEPTUAL

- Editor Vim
  - Vim es un editor de texto muy eficiente para programar en consola que no tiene nada que envidiar a los editores gráficos.

- Instalar Vim en
  - GNU/Linux

```
apt-get install vim
```

  - MS Windows
    - Para descarga en sistemas MS Windows `https://www.vim.org/download.php`
  - MacOS

```
brew install macvim
```

- Git
  - Git es un sistema de control de versiones creado por Linus Torvalds. Es eficiente, confiable.
- Instalar Git
  - GNU/Linux

```
apt-get install git
```

  - MS Windows
    - Para descarga en sistemas MS Windows `https://git-scm.com/download/win`
  - MacOS
    - Para descarga en sistemas MacOS `https://git-scm.com/download/mac`

```
brew install git
```

3. URL de Repositorio Github

- URL del Repositorio GitHub para clonar o recuperar.
- `https://github.com/rescobedoq/programacion.git`
- URL para el laboratorio 01 en el Repositorio GitHub.
- `https://github.com/rescobedoq/programacion/tree/main/lab01`

4. Resolucion de la tarea

4.1. Explicacion del Codigo1

```
<!DOCTYPE html>
<html lang="es">
  <head>
    <title></title>
    <link rel="stylesheet" href="estilo.css">
    <meta charset="UTF-8">
    <script src="teclado.js"></script>
  </head>

  <body>

        <div class="general">
    <table class="tabla">
      <tr>
              <td id="tabla"><div id="izq"><label>Seleccione:</label></div></td>
      <th colspan="2">
              <div class="select1">
              <select id="input">
                      <option>Multired Global Debito</option>
                      <option>Multired Country Debito</option>
                      <option>Multired Center Debito</option>
                      </select>
      </th>
      </tr>
      <tr>
              <td id="tabla"><div id="izq"><label> Numero de tarjeta:</label></div></td>
        <th colspan="2" id="i"><input id="number" type="number"></th>
      </tr>
      <tr>
              <td id="tabla"><div id="izq"><label> Tipo y Nº Documento:</label></div></td>
        <td>
                <div class="select">
              <select id="medio">
                      <option>DNI</option>
                      <option>Pasaporte</option>
                      <option>Carnet de Extranjeria</option>
                      </select>
                      </div>
```

```
                    </div>
          </td>
          <td><input id="medio" type="number"></td>
        </tr>
        <tr>
              <td width="160px" align="right"><label>Ingresa tu clave usando el teclado virtual:</label></td>
          <td>
            <table>
              <tr>
                <td><input class="button" id="uno" type="button" onclick="Ingreso('uno')"></td>
                <td><input class="button" id="dos" type="button" onclick="Ingreso('dos')"></td>
                <td><input class="button" id="tres" type="button" onclick="Ingreso('tres')"></td>
              </tr>
              <tr>
                <td><input class="button" id="cuatro" type="button" onclick="Ingreso('cuatro')"></td>
                <td><input class="button" id="cinco" type="button" onclick="Ingreso('cinco')"></td>
                <td><input class="button" id="seis" type="button" onclick="Ingreso('seis')"></td>
              </tr>
              <tr>
                <td><input class="button" id="siete" type="button" onclick="Ingreso('siete')"></td>
                <td><input class="button" id="ocho" type="button" onclick="Ingreso('ocho')"></td>
                <td><input class="button" id="nueve" type="button" onclick="Ingreso('nueve')"></td>
              </tr>
              <tr>
                <td><input class="button" id="cero" type="button" onclick="Ingreso('cero')"></td>
                <th colspan=2><input id="limpiar" type="button" onclick="Limpiar()" value="limpiar" ></th>
              </tr>
            </table>
          </td>
          <td>
                <div id="link">
                <img src="contraseña.png" height="25" width="28">
                      <a href="">Genera tu Clave de internet</a>
                </div>
                <br><div style="width:150px">Ingresa tu clave de Internet(06 digitos)</div>
          <br>
        <!--input id="contra" type="password" readonly-->
        <input id="contra" type="text" readonly>
              <br>
              <div id="link">
              <img src="rojo.png" width="25" height="25">
```

```
                <div id="link">
                <img src="contraseña.png" height="25" width="28">
                      <a href="">Genera tu Clave de internet</a>
                </div>
                <br><div style="width:150px">Ingresa tu clave de Internet(06 digitos)</div>
            <br>
        <!--input id="contra" type="password" readonly-->
        <input id="contra" type="text" readonly>
              <br>
              <div id="link">
              <img src="rojo.png" width="25" height="25">
              <a href="">Olvide mi clave</a>
              </div>
            </td>
          </tr>
          <tr>
                <td align="right"><label>Ingresa el texto de la imagen:</label></td>
            <td>
            <div id="inletter">
              <img src="pepe.jpg">
              <div id="letter">
              <img src="reset.png" width="20" height="20">
              <label>Cambiar texto</label>
              </div>
            </div>
            </td>
            <td><input id="medio" type="text"</td>
          </tr>
        </table>
        </div>
      </body>
    </html>
```

En este archivo cree una estructura html, con etiquetas de input, div, table y entre otros. La tabla la utilize para poder simular el div, de cada parte; por lo que entonces, me centre en atributos de cada elemento para acomodar los botones.

```
#input{
  border: 0.5px solid;
  border-radius: 3px;
  width: 242px;
  display:flex;
}
#number{
  border: 0.5px solid;
  border-radius: 3px;
  width: 235px;
  display:flex;
}
#form_general{
  text-align: center;
}
.general{
        display:flex;
        justify-content:center;
}
#tabla{
        text-align:right;
}
.tabla{
}
```

```
25      #medio{
26        border: 0.5px solid;
27        border-radius: 3px;
28       width: 100px;
29      }
30      .button{
31        border: 0.5px solid;
32        border-radius: 3px;
33        width: 31px;
34        height: 31px;
35      }
36      #limpiar{
37        border: 0.5px solid;
38        border-radius: 3px;
39        height: 31px;
40        width: 65px;
41      }
```

```
42      #link{
43      display: flex;
44      align-content: center;
45      }
46      #letter{
47      display: flex;
48      align-content:center;
49      }
50      #contra{
51          border: 0.5px solid;
52        border-radius: 3px;
53       width: 100px;
54      }
```

Finalmente el archivo teclado.js, es la programacion, del cambio de valor en el boton del teclado

```
1   document.addEventListener('DOMContentLoaded',function()
2   {
3   var lista = [0,1,2,3,4,5,6,7,8,9];
4   lista = lista.sort(function() {return Math.random() - 0.5});
5    document.getElementById("uno").value=lista[0];
6    document.getElementById("dos").value=lista[1];
7    document.getElementById("tres").value=lista[2];
8    document.getElementById("cuatro").value=lista[3];
9    document.getElementById("cinco").value=lista[4];
10   document.getElementById("seis").value=lista[5];
11   document.getElementById("siete").value=lista[6];
12   document.getElementById("ocho").value=lista[7];
13   document.getElementById("nueve").value=lista[8];
14   document.getElementById("cero").value=lista[9];})
15  function Ingreso(caracter){
16  document.getElementById("contra").value+=document.getElementById(caracter).value;
17  }
18  function Limpiar(){
19    document.getElementById("contra").value='';
20  }
21
```

En estas lineas, genero una variable local, que contenga un array con todos los numeros del teclado, para luego con la funcion list.sort, con la ubicacion generada por la funcion directa de Math.random()-0.5, genera la distribucion del nuevo teclado; finalmente con las lineas de document.getElementById, podemos acceder al elemento button, y cambiar su valor con la funcion value

4.2. Explicacion del codigo 2

Listing 1: app.use

```
app.use(express.static('public'));
  app.use(express.urlencoded({extended: true}));
```

la contante app, en la primera linea establece ala carpeta public como recurso para su ejecucion.Y la siguiente estable el enlace POST.

```
10  app.get('/', (req, res) => {
11    res.sendFile(__dirname + '/public/index.html');
12  });
13
14  // Ruta para ver todos los eventos
15  app.get('/eventos', (req, res) => {
16    fs.readdir('./eventos', (err, files) => {
17      if (err) {
18        console.error(err);
19        res.status(500).send('Error al leer los eventos');
20      } else {
21        const eventos = files.map((file) => file.split('.')[0]);
22        res.send(eventos);
23      }
24    });
25  });
```

En las lineas 10 a la 12 se hace llamado a la rama principal que hace imprime el index.html En las lineas 15 a la 25 nos refrencia en la direccion '/eventos', para imprimir el listado de elementos en el elemento './eventos' que se encuentra como argumento en la funcion de fs.readdir()

```
40  // Ruta para crear un evento
41  app.post('/eventos', (req, res) => {
42    const titulo = req.body.titulo;
43    const descripcion = req.body.descripcion;
44
45    fs.access(`./eventos/${titulo}.txt`, fs.constants.F_OK, (err) => {
46      if (!err) {
47        res.status(409).send('El evento ya existe');
48      } else {
49        const eventoData = `${titulo}\n${descripcion}`;
50
51        fs.writeFile(`./eventos/${titulo}.txt`, eventoData, 'utf8', (err) => {
52          if (err) {
53            console.error(err);
54            res.status(500).send('Error al crear el evento');
55          } else {
56            res.redirect('/');
57          }
58        });
59      }
60    });
61  });
```

En las lineas 41 a la 43 nos refrencia al metodo POST, crear un archivo en el listado de elementos, definimos una variable como titulo, que es el argumento que nos envian; de igual manera la descripcion. (45-61) se ubica en la carpeta de './eventos' para luego entrar en el archivo 'titulo', y verificar su existencia:

- **TRUE**, le envia el mensaje: .El evento ya existe"
- **FALSE**, concatena el titulo y la descripcion con un salto de linea
  - ejecuta la funcion de fs.write, para crear el archivo, agregarle el contenido (Titulo-descripcion)
    - **TRUE**, lo dirige a la rama principal
    - **FALSE**, imprime .Error al crear el evento"

```
// Ruta para eliminar un evento
app.delete('/eventos/:evento', (req, res) => {
  const evento = req.params.evento;
  fs.unlink(`./eventos/${evento}.txt`, (err) => {
    if (err) {
      console.error(err);
      res.status(500).send('Error al eliminar el evento');
    } else {
      res.send('Evento eliminado exitosamente');
    }
  });
});

// Iniciar el servidor
app.listen(port, () => {
  console.log(`La aplicaci�n est� escuchando en el puerto ${port}`);
});
```

En las lineas 4de app.delete remueve la —ruta de ./eventos/:eventos; siendo ':evento' la variable del archivo, y para ello la definimos esta variable dentro de la funcion, con la constante evento.
Luego la funcion fs.unlink remueve el archivo en la direccion "./eventos/evento.txt"

- **TRUE**, ejecuta la linea que imprimr .Evento eliminado exitosamente"
- **FALSE**, imprime .Error al eliminar el evento"

Finalmente la linea conocida para determinar el puerto del servidor y su ejecucion

4.3. ./public/script.js

```
document.addEventListener('DOMContentLoaded', () => {
const crearEventoForm = document.getElementById('crearEventoForm');
const eventosList = document.getElementById('eventosList');

// Cargar eventos al iniciar
cargarEventos();

// Enviar formulario para crear evento
crearEventoForm.addEventListener('submit', (e) => {
  e.preventDefault();
  const titulo = document.getElementById('titulo').value;
  const descripcion = document.getElementById('descripcion').value;
  crearEvento(titulo, descripcion);
});

// Cargar eventos
function cargarEventos() {
  fetch('/eventos')
    .then((response) => response.json())
    .then((data) => {
      eventosList.innerHTML = '';
      data.forEach((evento) => {
        const li = document.createElement('li');
        li.innerText = evento;
        eventosList.appendChild(li);
      });
    })
    .catch((error) => console.error(error));
}
```

```
// Crear evento
function crearEvento(titulo, descripcion) {
  fetch('/eventos', {
    method: 'POST',
    headers: {
      'Content-Type': 'application/x-www-form-urlencoded',
    },
    body: `titulo=${encodeURIComponent(titulo)}&descripcion=${encodeURIComponent(descripcion)}`,
  })
    .then((response) => {
      if (response.ok) {
        cargarEventos();
        crearEventoForm.reset();
      } else {
        throw new Error(response.statusText);
      }
    })
    .catch((error) => console.error(error));
}
});
```

En este archivo declaro las funciones del archivo html para direccionar a una nueva ruta, y de la misma manera vincular las elementos impregnados de una funcion POST

4.4. Estructura de laboratorio 03

- El contenido que se entrega en este laboratorio es el siguiente:

```
lab2/
|--- LATEX/
    |   |--- main.tex
    |   |--- lab3.pdf
    |   |--- img/
        |   |--- Calculadora.png
        |   |--- indexs1.png
        |   |--- indexs2.png
        |   |--- indexs3.png
        |   |--- indexs4.png
        |   |--- logo_abet.png
        |   |--- logo_episunsa.png
        |   |--- logo_unsa.png
        |   |--- parte1.1.png
        |   |--- parte1.2.png
        |   |--- parte1.3.png
        |   |--- script1.png
        |   |--- script2.png
        |   |--- style1.png
        |   |--- style2.png
        |   |--- style3.png
        |   |--- teclado.png
|--- parte1/
    |   |--- Ejercicio1.html
    |   |--- contrasea.png
    |   |--- estilo.css
    |   |--- pepe.jpg
    |   |--- reset - copia.png
    |   |--- reset.png
    |   |--- rojo.png
    |   |--- teclado.js
|--- parte2/
    |   |--- calculadora.html
    |   |--- funcion.js
    |   |--- style.css
|--- partes_de_prueba/
    |   |--- Ejercicio1.2.html
    |   |--- prototipo de CSS
```

5. Cuestionario

- Pruebe este código de arrayGenerator() en la página https://jslint.com/
- Revisar esta discusión en stackoverflow - `https://stackoverflow.com/questions/4852017/how-to-initialize-an-arrays-length-in-javascript`
    - ¿Cómo se pueden resolver los warnings?
    - ¿Se puede modificar la solución usando map? ¿Cómo?

6. Rúbricas

6.1. Entregable Informe

Tabla 1: Tipo de Informe

Informe	
Latex	El informe está en formato PDF desde Latex, con un formato limpio (buena presentación) y facil de leer.

6.2. Rúbrica para el contenido del Informe y demostración

- El alumno debe marcar o dejar en blanco en celdas de la columna **Checklist** si cumplio con el ítem correspondiente.
- Si un alumno supera la fecha de entrega, su calificación será sobre la nota mínima aprobada, siempre y cuando cumpla con todos lo items.
- El alumno debe autocalificarse en la columna **Estudiante** de acuerdo a la siguiente tabla:

Tabla 2: Niveles de desempeño

	Nivel			
Puntos	Insatisfactorio 25 %	En Proceso 50 %	Satisfactorio 75 %	Sobresaliente 100 %
2.0	0.5	1.0	1.5	2.0
4.0	1.0	2.0	3.0	4.0

Tabla 3: Rúbrica para contenido del Informe y demostración

Contenido y demostración		Puntos	Checklist	Estudiante	Profesor
1. GitHub	Hay enlace URL activo del directorio para el laboratorio hacia su repositorio GitHub con código fuente terminado y fácil de revisar.	2	X	2	
2. Commits	Hay capturas de pantalla de los commits más importantes con sus explicaciones detalladas. (El profesor puede preguntar para refrendar calificación).	4	X	2	
3. Código fuente	Hay porciones de código fuente importantes con numeración y explicaciones detalladas de sus funciones.	2	X	1	
4. Ejecución	Se incluyen ejecuciones/pruebas del código fuente explicadas gradualmente.	2	X	1	
5. Pregunta	Se responde con completitud a la pregunta formulada en la tarea. (El profesor puede preguntar para refrendar calificación).	2	X	2	
6. Fechas	Las fechas de modificación del código fuente estan dentro de los plazos de fecha de entrega establecidos.	2	X	1	
7. Ortografía	El documento no muestra errores ortográficos.	2	X	2	
8. Madurez	El Informe muestra de manera general una evolución de la madurez del código fuente, explicaciones puntuales pero precisas y un acabado impecable. (El profesor puede preguntar para refrendar calificación).	4	X	1	
Total		20		12	

7. Referencias

- https://www.w3schools.com/java/default.asp
- https://www.geeksforgeeks.org/insertion-sort/
- https://github.com/WillianAyamamani/pw2-lab-d-23a/tree/main/lab1